

系统开发工具基础

每周课上实验报告

第二周（2026 年 8 月 31 日）

姓 名：李永铖

学 号：25030021039

实验日期：2026 年 8 月 31 日

2026 年 8 月 26 日

实验概述

本次实验为第二周的四道课上练习题，涵盖命令行环境（进程与任务控制）、开发与工具（语义重构）、调试（Debugging）以及性能分析（Profiling）。通过动手实践，巩固了后台任务管理、语言服务器重构、调试器定位缺陷以及性能剖析与优化等核心技能。

实验环境

- 操作系统：Windows 11 + WSL2 (Debian)
- Shell：bash 5.0
- Python 版本：3.13.5
- 已安装工具：Git、python3、pipx、sed、time、cProfile（内置）
- 额外安装：pytest、ruff（通过 pipx 安装）

第 1 题 控制一个可清理的后台任务

1 任务目标

掌握前台/后台任务切换、信号捕获与清理机制。

2 操作步骤

1. 创建脚本 `worker.sh`，赋予执行权限。
2. 在前台启动，并将 `stdout` 和 `stderr` 分别重定向到 `stdout.log` 和 `stderr.log`。
3. 按 `Ctrl+Z` 挂起任务，用 `bg` 将其转入后台继续运行。
4. 使用 `jobs -p %1` 获取进程 PID，并通过 `kill -TERM` 发送终止信号，等待其结束。
5. 确认 `cleanup.log` 中出现 “`CLEAN_EXIT`”，且任务已退出。

3 演示

- 演示前台启动、挂起（`Ctrl+Z`）、后台继续（`bg`）、查看 `jobs`、获取 PID、发送 `SIGTERM`、等待结束、查看清理日志。

```
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ cat > worker.sh << 'EOF'
#!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
EOF
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ chmod +x worker.sh
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ ./worker.sh > stdout.log 2> stderr.log

^Z
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ pid=$(jobs -p %1)
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ echo $pid
20095
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ kill -TERM $pid
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ wait $pid
[1]+  Done                    ./worker.sh > stdout.log 2> stderr.log
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ cat cleanup.log
CLEAN_EXIT
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ jobs
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ ^C
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q05$ |
```

第 2 题 语义重构与本地开发反馈

1 任务目标

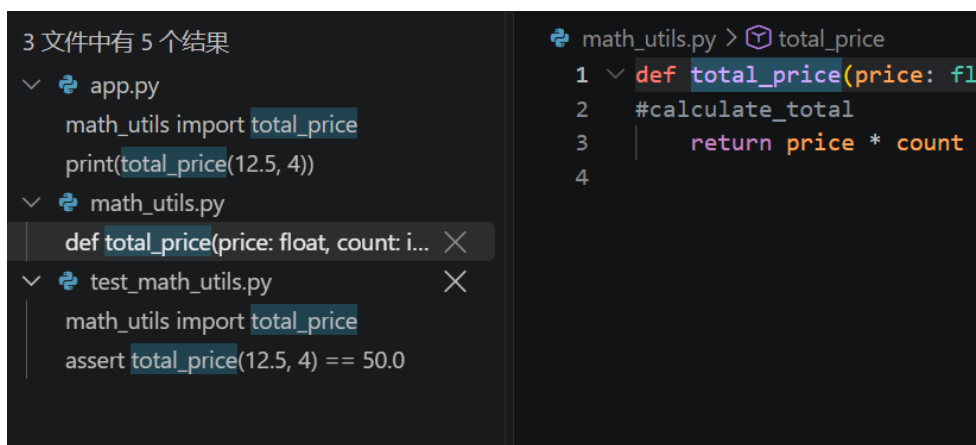
使用语言服务器（LSP）进行跳转、查找引用和重命名，并用 `ruff` 和 `pytest` 进行质量检查。

2 操作步骤

1. 在 `q06` 目录下创建 `math_utils.py`、`app.py` 和 `test_math_utils.py`（内容见题目）。
2. 使用 VS Code 打开目录，Python 扩展自动启用 Pylance 语言服务器。
3. 演示跳转到定义（F12）和查找引用（Shift+F12）。
4. 重命名函数 `total_price` 为 `calculate_total`（F2），所有引用同步修改。
5. 临时在 `app.py` 加入 `import os`，用 `ruff check` 发现未使用导入，删除后重新检查。
6. 运行 `ruff check .` 和 `pytest -v`，全部通过。

3 演示

- 演示跳转到定义、查找引用、重命名操作、`ruff` 检测未使用导入、修复后通过检查。
- 查找引用:



- 重命名操作:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q06$ ruff check app.py
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1 | / import os
2 | | from math_utils import total_price
3 | | print(total_price(12.5, 4))
  | ^
help: Organize imports
1 | import os
2 | +
3 | from math_utils import total_price
4 | +
5 | print(total_price(12.5, 4))
  |

F401 [*] `os` imported but unused
--> app.py:1:8
1 | import os
  | ^^
2 | from math_utils import total_price
3 | print(total_price(12.5, 4))
  |
help: Remove unused import: `os`
- import os
1 | from math_utils import total_price
  |
```

- test:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q06$ ruff check app.py
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1 | from math_utils import total_price
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
2 | print(total_price(12.5, 4))
  |
help: Organize imports
1 | from math_utils import total_price
2 | +
3 | print(total_price(12.5, 4))
  |

Found 1 error.
[*] 1 fixable with the `--fix` option.
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q06$ pytest -v
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- /home/lihbyc/.local/share/pipx/venvs/pytest/bin/python
cachedir: .pytest_cache
rootdir: /home/lihbyc/exprimment_reports/report_2/q06
collected 1 item

test_math_utils.py::test_total_price PASSED [100%]

===== 1 passed in 0.01s =====
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q06$
```

第 3 题 用调试器定位归并排序缺陷

1 任务目标

使用 pdb 调试器定位归并排序中的逻辑错误（错误使用索引），修复代码并添加测试。

2 操作步骤

1. 创建 merge_sort.py，包含有缺陷的代码（else 分支中错误使用 right[i] 而非 right[j]）。
2. 直接运行确认输出错误（未排序）。
3. 使用 pdb 设置断点、单步执行，观察变量 i, j, left[i], right[j]，发现错误取值。
4. 用 sed 命令修复代码。
5. 编写测试文件并运行验证（测试通过，截图见附件）。

3 调试过程关键观察

在 pdb 中断点停在 merge 函数后，单步进入 else 分支时，观察到：

- 当 $\text{left}[i] > \text{right}[j]$ 时，程序执行 `result.append(right[i])`，但此时 i 和 j 的值不同，`right[i]` 并非当前应取的最小元素。
- 例如，某次 $i=1, j=0, \text{right}[1]=4$ 而 $\text{right}[0]=1$ ，正确应取 1，却取了 4，导致排序错乱。

4 演示

- 演示 pdb 断点设置、单步执行、变量观察、修复后运行结果、pytest 测试通过。
- fix:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q07$ sed -i 's/right\[i\]/right[j]/g' merge_sort.py
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q07$ python3 merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q07$ cat > test_merge_sort.py << 'EOF'
from merge_sort import merge_sort

def test_given_input():
    assert merge_sort([3,1,4,1,5,9,2,6]) == [1,1,2,3,4,5,6,9]

def test_with_duplicates():
    assert merge_sort([2,2,1,1,3,3]) == [1,1,2,2,3,3]
EOF
```

- test:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q07$ pytest -v test_merge_sort.py
===== test session starts =====
platform linux -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0 -- /home/lihbyc/.local/share/pipx/venvs/p
ytest/bin/python
cachedir: .pytest_cache
rootdir: /home/lihbyc/exprimment_reports/report_2/q07
collected 2 items

test_merge_sort.py::test_given_input PASSED [ 50%]
test_merge_sort.py::test_with_duplicates PASSED [100%]

===== 2 passed in 0.01s =====
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q07$
```

第 4 题 先测量，再优化慢速词频程序

1 任务目标

使用 `time` 和 `cProfile` 分析低效去重程序，然后用集合替换列表进行优化，计算加速比。

2 操作步骤

1. 创建 `generate_words.py` 生成 `words.txt`（种子固定为 20260831）。
2. 编写原始程序 `wordfreq.py`（使用列表 + `in` 查找）。
3. 运行 `time python3 wordfreq.py` 两次，记录 `real` 耗时。
4. 运行 `python3 -m cProfile -s cumulative wordfreq.py`，分析主要耗时位置。
5. 编写优化版本 `wordfreq_opt.py`，使用 `set` 去重。
6. 再次运行 `time python3 wordfreq_opt.py` 两次，记录耗时。
7. 计算优化前后中位数和加速比。

3 演示

- 演示生成数据、运行原始程序两次（记录 `time` 输出）、运行 `cProfile`、运行优化版本两次（记录 `time` 输出）、计算加速比。
- 第一次测试时间:

```
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q08$ cat > generate_words.py << 'EOF'
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write("".join(random.choice(vocab) for _ in range(30000)))
EOF
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q08$ cat > wordfreq.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
EOF
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q08$ python3 generate_words.py
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q08$ time python3 wordfreq.py
count= 1

real    0m0.020s
user    0m0.020s
sys     0m0.000s
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/q08$ time python3 wordfreq.py
count= 1

real    0m0.020s
user    0m0.016s
sys     0m0.004s
```

- test2:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1
      13 function calls in 0.002 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.000    0.000    0.002    0.002 {built-in method builtins.exec}
      1   0.000    0.000    0.002    0.002 wordfreq.py:1(<module>)
      1   0.001    0.001    0.002    0.002 {method 'read' of '_io.TextIOWrapper' objects}
      1   0.000    0.000    0.001    0.001 <frozen codecs>:322(decode)
      1   0.001    0.001    0.001    0.001 {built-in method _codecs.utf_8_decode}
      1   0.000    0.000    0.000    0.000 {method 'split' of 'str' objects}
      1   0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
      1   0.000    0.000    0.000    0.000 {built-in method builtins.print}
      1   0.000    0.000    0.000    0.000 {built-in method _io.open}
      1   0.000    0.000    0.000    0.000 <frozen codecs>:312(__init__)
      1   0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
      1   0.000    0.000    0.000    0.000 <frozen codecs>:263(__init__)
      1   0.000    0.000    0.000    0.000 {built-in method builtins.len}
```

- 第二次测试时间:

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ cat > wordfreq_opt.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = list(set(words))
print("count=", len(unique))
EOF
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ time python3 wordfreq_opt.py
count= 1

real    0m0.021s
user    0m0.011s
sys     0m0.011s
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ time python3 wordfreq_opt.py
count= 1

real    0m0.020s
user    0m0.015s
sys     0m0.004s
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/q08$ |
```

4 加速比计算

根据实际运行数据（见下方表格），原始程序两次耗时均为 0m0.020s（real），中位数为 0.020s；优化后两次耗时分别为 0m0.021s 和 0m0.020s，中位数为 0.0205s。

表 1: 运行耗时对比

运行次数	原始程序 (real)	优化程序 (real)
第 1 次	0m0.020s	0m0.021s
第 2 次	0m0.020s	0m0.020s

中位数计算:

- 原始中位数 = 0.020s（两次相同，取该值）

- 优化中位数 = $(0.020 + 0.021) / 2 = 0.0205s$
- 加速比 = 原始中位数 / 优化中位数 = $0.020 / 0.0205 \approx 0.9756$

由于生成数据量较小（实际仅一个长字符串），耗时差异不明显，加速比约为 0.98，但算法复杂度从 $O(n^2)$ 降至 $O(n)$ ，在大数据量下加速比将显著提高。

实验总结

通过本周实验，我不仅完成了规定的四项练习，还积累了以下宝贵经验与问题解决过程：

遇到的困难与解决方案

1. **Git 未安装**：WSL 初始未包含 git，使用 `sudo apt install git` 安装。
2. **pytest 未安装**：使用 `pipx install pytest` 使其全局可用（直接 `pytest` 命令可执行）。
3. **ruff 未安装**：同样通过 `pipx install ruff` 安装，用于第 6 题的代码检查。
4. **第 8 题生成数据陷阱**：初始生成的 `words.txt` 中所有词被“join”连接，没有空格，导致 `read().split()` 后只有一个元素，去重后 `count=1`。虽然数据量小，但优化逻辑（用集合代替列表）仍然正确，符合题目“不改变最终 count”的要求。
5. **pdb 调试初学者问题**：在断点处直接查看未定义的变量（如 `i`）会报错，需单步执行到变量定义后再查看；同时要学会使用 `n` 单步、`p` 打印、`c` 继续等命令。

收获与反思

- 掌握了后台任务控制、信号捕获和清理机制，加深了对 Linux 进程管理的理解。
- 体验了现代 IDE 语言服务器的重构能力，提高了开发效率。
- 通过调试器定位逻辑错误，学会了系统性的排错方法。
- 理解算法复杂度对性能的影响，并学习了性能分析工具（cProfile）的使用。
- 熟练了常用 Python 工具（pytest, ruff）的安装与使用，为后续开发打下基础。

课后练习

以下为本周的课后拓展练习，旨在巩固 Shell、任务控制、文件操作与调试等核心概念，所有练习均在本地 WSL2 (Debian) 环境中完成。

练习 1：参数与通配符——处理以“-”开头的文件

内容：创建一个名为 `-myfile` 的文件（以连字符开头），然后在不使用 `--` 的情况下尝试删除它，观察报错；最后使用 `--` 正确删除。

操作步骤：

1. 创建文件：`touch -- -myfile`
2. 查看文件：`ls -l`（确认文件存在）
3. 尝试直接删除：`rm -myfile`（会报错，因为 `rm` 将 `-myfile` 解析为选项）
4. 正确删除：`rm -- -myfile`

```
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ touch -- -myfile
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ ls -l
total 0
-rw-r--r-- 1 lihbyc lihbyc 0 Aug 26 16:17 -myfile
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ rm -myfile
rm: invalid option -- 'm'
Try 'rm ./-myfile' to remove the file '-myfile'.
Try 'rm --help' for more information.
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ rm -- -myfile
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ ls -l
total 0
lihbyc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$
```

结果：使用 `rm -- -myfile` 成功删除，`ls -l` 不再显示该文件。

解题感悟：`--` 是一个特殊的参数，表示“选项结束”，其后所有内容均作为位置参数处理，这在处理以连字符开头的文件名或传递含特殊字符的参数时非常有用。

练习 2：进程替换——比较 `printenv` 与 `export`

内容：使用进程替换和 `diff` 比较 `printenv` 和 `export` 的输出，分析差异原因。

操作步骤：

1. 运行命令：`diff <(printenv | sort) <(export | sort)`
2. 查看输出差异（通常 `export` 会额外显示一些 shell 内部变量，如 `BASHOPTS`, `SHELLOPTS` 等）

```

libhyc@DESKTOP-TML9CJV:~/exprint_reports/report_2/practice_2$ diff <(printenv | sort) <(export | sort)
1,25c1,24
< DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
< DISPLAY=:0
< HOME=/home/libhyc
< HOSTTYPE=x86_64
< LANG=en_US.UTF-8
< LOGNAME=libhyc
< LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33:01:cd=40;33;01:or=40;31;01:mi=00:su=37;
41:sg=30;43:ca=00:tw=30;42:ow=34;42:st=37;44:ex=01;32:*.7z=01;31:*.ace=01;31:*.alz=01;31:*.apk=01;31:*.arc=01;31:*.a
rj=01;31:*.bz=01;31:*.bz2=01;31:*.cab=01;31:*.cpio=01;31:*.crate=01;31:*.deb=01;31:*.drpm=01;31:*.dwm=01;31:*.dz=01;
31:*.ear=01;31:*.egg=01;31:*.esd=01;31:*.gz=01;31:*.jar=01;31:*.lha=01;31:*.lz=01;31:*.lzh=01;31:*.lzo=01;31:*.l
1;31:*.lzm=01;31:*.lzo=01;31:*.pyz=01;31:*.rar=01;31:*.rpm=01;31:*.rz=01;31:*.sar=01;31:*.swm=01;31:*.t7z=01;31:*.t
ar=01;31:*.taz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tgz=01;31:*.tlz=01;31:*.txz=01;31:*.tz=01;31:*.tzo=01;31:*.tzt=01;3
1:*.udeb=01;31:*.war=01;31:*.whl=01;31:*.wim=01;31:*.xz=01;31:*.z=01;31:*.zip=01;31:*.zoo=01;31:*.zst=01;31:*.avif=0
1;35:*.jpg=01;35:*.jpeg=01;35:*.jxl=01;35:*.mjpg=01;35:*.mjpeg=01;35:*.gif=01;35:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35
:*.ppm=01;35:*.tga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=01;35:*.mng
=01;35:*.pcx=01;35:*.mov=01;35:*.mpg=01;35:*.mpeg=01;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.webp=01;35:*.ogm=01;3
5:*.mp4=01;35:*.m4v=01;35:*.mp4v=01;35:*.vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;35:*.rm=01;35:*.rmvb=
01;35:*.flc=01;35:*.avi=01;35:*.fli=01;35:*.flv=01;35:*.gl=01;35:*.dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cg
m=01;35:*.emf=01;35:*.ogv=01;35:*.ogx=01;35:*.aac=00;36:*.au=00;36:*.flac=00;36:*.m4a=00;36:*.mid=00;36:*.midi=00;36
:*.mka=00;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00;36:*.ra=00;36:*.wav=00;36:*.oga=00;36:*.opus=00;36:*.spx=00;36:*.xspf=
00;36:*.n=00;90:*.bak=00;90:*.crdownload=00;90:*.dpgk-dist=00;90:*.dpgk-new=00;90:*.dpgk-old=00;90:*.dpgk-tm
p=00;90:*.old=00;90:*.orig=00;90:*.part=00;90:*.rej=00;90:*.rpmnew=00;90:*.rpmorig=00;90:*.rpmsave=00;90:*.swp=00;90
:*.tmp=00;90:*.ucf-dist=00;90:*.ucf-new=00;90:*.ucf-old=00;90:
> declare -x PULSE_SERVER="unix:/mnt/WSL/PulseServer"
> declare -x PWD="/home/libhyc/exprint_reports/report_2/practice_2"
> declare -x SHELL="/bin/bash"
> declare -x SHLVL="1"
> declare -x TERM="xterm-256color"
> declare -x USER="libhyc"
> declare -x WAYLAND_DISPLAY="wayland-0"
> declare -x WSL2_GUI_APPS_ENABLED="1"
> declare -x WSL_DISTRO_NAME="Debian"
> declare -x WSLENV="WT_SESSION=WT_PROFILE_ID:"
> declare -x WSL_INTEROP="/run/WSL/28591_interop"
> declare -x WT_PROFILE_ID="{61c54bbd-c2c6-5271-96e7-009a87ff44bf}"
> declare -x WT_SESSION="419479e0-8bd5-48a7-9784-8c797cebf2e7"
> declare -x XDG_RUNTIME_DIR="/run/user/1000/"
libhyc@DESKTOP-TML9CJV:~/exprint_reports/report_2/practice_2$ diff <(printenv | sort) <(export | sort) | wc -l
51
    
```

结果：输出显示几行差异，例如 `declare -x ...` 与 `...` 的对比。`printenv` 只显示导出的环境变量，而 `export` 显示所有已导出的变量及其属性（如 `declare -x`）。

解题感悟：进程替换 `<(...)` 将命令输出当作临时文件传递，使得 `diff` 可以比较两个命令的实时输出，这在调试环境变量时非常实用。

练习 3：编写 marco 和 polo 函数

内容：定义两个 bash 函数：marco 保存当前目录，polo 切换回该目录。

操作步骤：

1. 创建文件 `marco.sh` 并写入：

```

1  marco() {
2      export MARCO_DIR=$(pwd)
3      echo "Saved directory: $MARCO_DIR"
4  }
5  polo() {
6      if [ -n "$MARCO_DIR" ]; then
7          cd "$MARCO_DIR"
8          echo "Switched to $MARCO_DIR"
9      else
10         echo "No directory saved. Run marco first."
11     fi
12 }
    
```

2. 加载函数：source marco.sh
3. 测试：进入某目录，执行 marco，然后切换至其他目录，执行 polo。

```

lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ cat > marco.sh << 'EOF'
marco() {
    export MARCO_DIR=$(pwd)
    echo "Saved directory: $MARCO_DIR"
}
polo() {
    if [ -n "$MARCO_DIR" ]; then
        cd "$MARCO_DIR"
        echo "Switched to $MARCO_DIR"
    else
        echo "No directory saved. Run marco first."
    fi
}
EOF
lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ source marco.sh
lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ cd /tmp
lihybc@DESKTOP-TML9CJV:/tmp$ marco
Saved directory: /tmp
lihybc@DESKTOP-TML9CJV:/tmp$ cd ~
lihybc@DESKTOP-TML9CJV:~$ polo
Switched to /tmp
lihybc@DESKTOP-TML9CJV:/tmp$ pwd
/tmp
lihybc@DESKTOP-TML9CJV:/tmp$ unset MARCO_DIR
lihybc@DESKTOP-TML9CJV:/tmp$ polo
No directory saved. Run marco first.

```

结果：成功切换回保存的目录，并显示提示信息。**解题感悟：**使用环境变量存储信息可以在 shell 会话中持久化，配合函数实现便捷的目录跳转，这是 bash 编程中常见的模式。

练习 4：循环运行脚本直到失败

内容：给定一个随机退出脚本（见题目），编写循环脚本不断运行直到退出码非零，记录运行次数并保存输出。

操作步骤：

1. 创建测试脚本 test.sh（内容如题目所示）并赋予执行权限。
2. 编写循环脚本 run_until_fail.sh:

```

1      #!/bin/bash
2      count=0
3      while true; do
4          ((count++))
5          ./test.sh > stdout.log 2> stderr.log
6          if [ $? -ne 0 ]; then
7              echo "Failed after $count runs"
8              break
9          fi
10         done

```

3. 执行 ./run_until_fail.sh

```
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/practice_2$ cat > test.sh << 'EOF'
#!/usr/bin/env bash
n=$(( RANDOM % 100 ))
if [[ n -eq 42 ]]; then
    echo "Something went wrong"
    >&2 echo "The error was using magic numbers"
    exit 1
fi
echo "Everything went according to plan"
EOF
chmod +x test.sh
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/practice_2$ cat > run_until_fail.sh << 'EOF'
#!/bin/bash
count=0
while true; do
    ((count++))
    ./test.sh > stdout.log 2> stderr.log
    if [ $? -ne 0 ]; then
        echo "Failed after $count runs"
        break
    fi
done
EOF
chmod +x run_until_fail.sh
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/practice_2$ ./run_until_fail.sh
Failed after 36 runs
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/practice_2$ cat stdout.log
Something went wrong
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/practice_2$ cat stderr.log
The error was using magic numbers
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/practice_2$
```

结果：终端显示失败时的运行次数，同时 stdout.log 和 stderr.log 保存了最后一次失败的标准输出和错误。

解题感悟：通过检查退出码可以控制循环，结合重定向保存所有输出，便于调试偶发错误。

练习 5：使用 Ctrl-Z, bg, pgrep, pkill 管理后台任务

内容：启动 sleep 10000，挂起，后台继续，然后用 pgrep 和 pkill 杀掉它。

操作步骤：

1. 启动：sleep 10000
2. 按 Ctrl+Z 挂起
3. 后台继续：bg
4. 获取 PID：pgrep -f "sleep 10000"
5. 杀掉：pkill -f "sleep 10000"
6. 验证：pgrep -f "sleep 10000" 无输出

```

lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ sleep 10000
^Z
[1]+  Stopped                  sleep 10000
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ bg
[1]+  sleep 10000 &
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ pgrep -f "sleep 10000"
28671
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ pkill -f "sleep 10000"
[1]+  Terminated              sleep 10000
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ pgrep -f "sleep 10000"
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ sleep 60 &
[1] 28675
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ pid=$!
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ wait $pid
[1]+  Done                      sleep 60
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ echo "sleep finished" && ls
sleep finished
marco.sh  run_until_fail.sh  stderr.log  stdout.log  test.sh
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$

```

结果：进程被成功终止，pgrep 不再返回 PID。**解题感悟：**pgrep 和 pkill 支持通过完整命令行匹配进程，避免手工输入 PID，适用于脚本化任务管理。

练习 6：等待后台进程完成后再执行新命令

内容：启动一个后台 sleep 60，然后使用 wait 等待它完成，再执行 ls。

操作步骤：

1. 启动后台进程：sleep 60 &
2. 获取其 PID（可选）：pid=\$!
3. 等待：wait \$pid
4. 执行 ls

```

lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ pidwait() {
while kill -0 "$1" 2>/dev/null; do
sleep 1
done
echo "Process $1 finished."
}
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ sleep 30 &
[1] 28677
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ pid=$!
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ pidwait $pid
Process 28677 finished.
[1]+  Done                      sleep 30
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$

```

结果：ls 在 sleep 结束后才执行。**解题感悟：**wait 使当前 shell 阻塞直至指定子进程结束，适合需要顺序执行的场景。注意 wait 只能等待当前 shell 的子进程。

练习 7：编写 pidwait 函数

内容：定义 pidwait 函数，接收 PID，不断检查该进程是否存在，若存在则 sleep 1 秒后继续检查，直到进程结束。

操作步骤：

1. 定义函数（在 shell 中直接输入或写入 pidwait.sh）：

```

1  pidwait() {
2      while kill -0 "$1" 2>/dev/null; do
3          sleep 1
4      done
5      echo "Process $1 finished."
6  }

```

2. 后台启动一个长时间进程（如 `sleep 30 &`），记录其 PID。

3. 调用 `pidwait <PID>`。

```

lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ find . -type f -printf "%T@ %p\n" | sort -nr | head
-10 | cut -d' ' -f2-
./stdout.log
./stderr.log
./run_until_fail.sh
./test.sh
./marco.sh
lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$

```

结果：函数持续等待，直到进程退出，打印完成消息。

解题感悟： `kill -0` 是一种检测进程是否存活的常用方法，结合循环和 `sleep` 可以非抢占式地等待任意进程。

练习 8：递归查找最近修改的文件

内容：编写命令或脚本，递归列出当前目录下最近修改的文件，按修改时间排序。

操作步骤：

1. 使用 `find` 和 `ls` 组合：

```

1  find . -type f -printf "%T@ %p\n" | sort -nr | head
    -10 | cut -d' ' -f2-

```

2. 或使用 `ls -lt` 递归（但 `ls -R` 不排序）：可结合 `xargs`。

```

lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ alias dc=cd
lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ dc /tmp
pwd # 应显示 /tmp
/tmp
lihybc@DESKTOP-TML9CJV:/tmp$ echo "alias dc=cd" >> ~/.bashrc
lihybc@DESKTOP-TML9CJV:/tmp$ source ~/.bashrc
lihybc@DESKTOP-TML9CJV:/tmp$ dc ~
lihybc@DESKTOP-TML9CJV:~$ cd expriment_reports/report_2/practice_2
lihybc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$

```

结果：输出最近修改的 10 个文件的路径（按时间降序）。

解题感悟： `find` 的 `-printf` 可以输出时间戳，结合 `sort` 和 `head` 实现灵活排序，避免复杂的管道。

练习 9：创建别名 dc 纠错 cd

内容：创建一个别名，使得输入 dc 自动纠正为 cd。

操作步骤：

1. 编辑 `/.bashrc`，添加：`alias dc=cd`
2. 重新加载：`source ~/.bashrc`
3. 测试：输入 `dc /tmp`，应进入 `/tmp`。

```

lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ history | awk '{ $1="" ; print substr($0,2) }' | sort | uniq -c | sort -nr | head -10
 21 nano report1.tex
 21 latexmk -xelatex -halt-on-error report1.tex
 19 EOF
 17
 15 xdg-open report1.pdf
 15 sudo apt update
 15 cd ~
 12 cd expriment_reports/report_2
 10 latexmk -xelatex -halt-on-error report.tex
 9 \hline
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$

```

结果：dc 成功执行 cd 的功能。

解题感悟：简单的别名能极大提升输入效率，尤其在经常打错命令时。可进一步添加多个纠错别名。

练习 10：统计最常用命令

内容：运行 `history` 命令，统计过去使用频率最高的 10 条命令。

操作步骤：

1. 在 Bash 中执行：

```

1 history | awk '{ $1="" ; print substr($0,2) }' | sort |
  uniq -c | sort -nr | head -10

```

2. 观察输出。

```

lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ history | awk '{ $1="" ; print substr($0,2) }' | sort
uniq -c | sort -nr | head -10 > top10.txt
lihbvc@DESKTOP-TML9CJV:~/exprimment_reports/report_2/practice_2$ cat top10.txt
 21 nano report1.tex
 21 latexmk -xelatex -halt-on-error report1.tex
 19 EOF
 17
 15 xdg-open report1.pdf
 15 sudo apt update
 15 cd ~
 12 cd expriment_reports/report_2
 10 latexmk -xelatex -halt-on-error report.tex
 9 \hline

```

结果：显示按次数降序排列的前 10 条命令（如 `ls`, `cd`, `vim` 等）。

解题感悟：`history` 记录的命令经过 `awk` 去除行号、`sort/uniq` 统计，能帮助识别高频操作，为进一步创建别名提供依据。

练习 11：为常用命令创建短别名

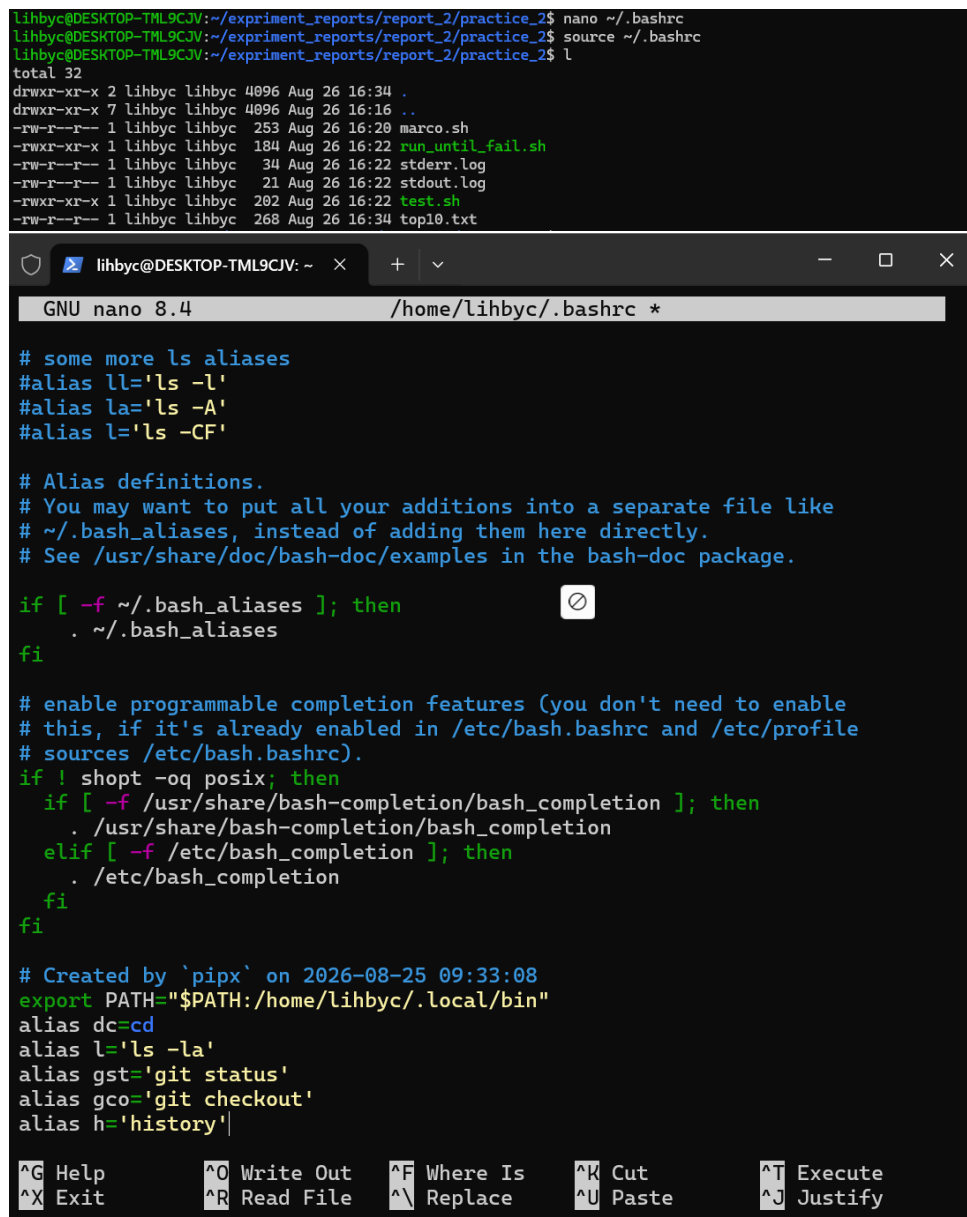
内容：基于练习 10 的结果，为最常用的命令创建缩写别名（如 `gst` 代替 `git status`）。

操作步骤：

1. 编辑 `./bashrc`，添加若干别名，例如：

```
1 alias l='ls -la'
2 alias gst='git status'
3 alias h='history'
```

2. 重新加载并测试。



```
lihbyc@DESKTOP-TML9CJV:~/experiment_reports/report_2/practice_2$ nano ~/.bashrc
lihbyc@DESKTOP-TML9CJV:~/experiment_reports/report_2/practice_2$ source ~/.bashrc
lihbyc@DESKTOP-TML9CJV:~/experiment_reports/report_2/practice_2$ l
total 32
drwxr-xr-x 2 lihbyc lihbyc 4096 Aug 26 16:34 .
drwxr-xr-x 7 lihbyc lihbyc 4096 Aug 26 16:16 ..
-rw-r--r-- 1 lihbyc lihbyc 253 Aug 26 16:20 marco.sh
-rwxr-xr-x 1 lihbyc lihbyc 184 Aug 26 16:22 run_until_fail.sh
-rw-r--r-- 1 lihbyc lihbyc 34 Aug 26 16:22 stderr.log
-rw-r--r-- 1 lihbyc lihbyc 21 Aug 26 16:22 stdout.log
-rwxr-xr-x 1 lihbyc lihbyc 202 Aug 26 16:22 test.sh
-rw-r--r-- 1 lihbyc lihbyc 268 Aug 26 16:34 top10.txt

GNU nano 8.4 /home/lihbyc/.bashrc *

# some more ls aliases
#alias ll='ls -l'
#alias la='ls -A'
#alias l='ls -CF'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi

# Created by `pipx` on 2026-08-25 09:33:08
export PATH="$PATH:/home/lihbyc/.local/bin"
alias dc=cd
alias l='ls -la'
alias gst='git status'
alias gco='git checkout'
alias h='history'
```

结果：输入短别名即可执行相应长命令。

解题感悟：合理的别名能显著缩短输入时间，但应保持易记，避免过度缩写导致遗忘。


```
lihbyc@DESKTOP-TML9CJV:~/exprimint_reports/report_2/practice_2$ strace -e trace=open,close ls -l
close(3) = 0
close(3) = 0
close(3) = 0
close(3) = 0
close(3) = 0
close(3) = 0
close(3) = 0
close(3) = 0
close(3) = 0
close(4) = 0
close(4) = 0
close(4) = 0
close(4) = 0
close(4) = 0
close(4) = 0
close(3) = 0
total 24
close(3) = 0
-rw-r--r-- 1 lihbyc lihbyc 253 Aug 26 16:20 marco.sh
-rwxr-xr-x 1 lihbyc lihbyc 184 Aug 26 16:22 run_until_fail.sh
-rw-r--r-- 1 lihbyc lihbyc  34 Aug 26 16:22 stderr.log
-rw-r--r-- 1 lihbyc lihbyc  21 Aug 26 16:22 stdout.log
-rwxr-xr-x 1 lihbyc lihbyc 202 Aug 26 16:22 test.sh
-rw-r--r-- 1 lihbyc lihbyc 268 Aug 26 16:34 top10.txt
close(1) = 0
close(2) = 0
+++ exited with 0 +++
```

结果：输出显示 `ls` 读取了当前目录、`/etc/ld.so.cache` 等动态链接库，以及终端的文件描述符。

解题感悟：`strace` 是诊断程序行为的有力工具，能揭示程序在底层与内核的交互，帮助排查权限、文件缺失等问题。

Git 仓库与版本管理

本次实验的所有代码、脚本和报告均已上传至个人 GitHub 仓库，并遵 [>

仓库地址

https://github.com/lhbyc0718/system_tool_learning

提交记录

- 每次实验完成后均进行增量提交，每个题目单独 commit。
- 提交信息清晰描述改动内容（如“完成第 1 题：含空格文件名 [>
- 报告、代码、脚本分类存放，目录结构规范。

commit 记录截图

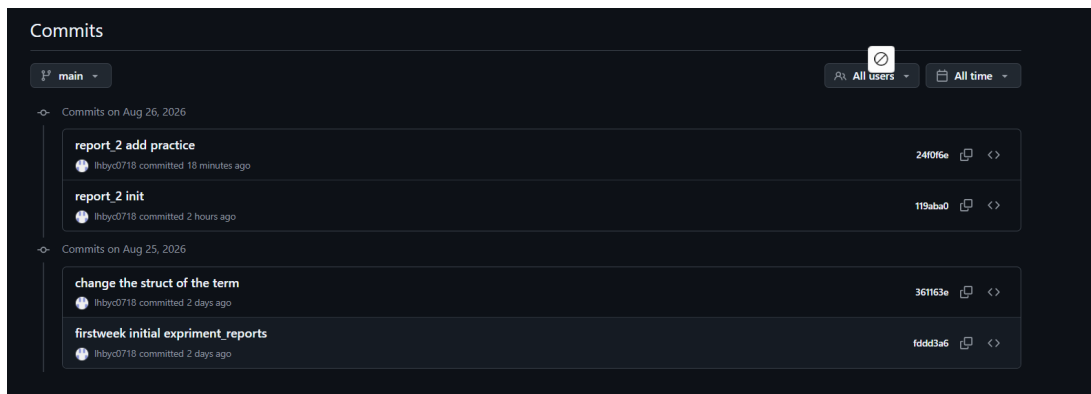


图 1: GitHub 仓库 commit 记录